AKTE 3 · ERMITTLUNG 3.1 · DS 5

Kassensturz

STADTFEST

NAME _____ DATUM _____

> **FALL-EINSTIEG** *Nach dem Stadtfest am 29. August fehlen in der Festkasse 312,50 €. Die Stände haben jeden Verkauf als Bon erfasst – Frauke Melchers vom Festausschuss hat uns alle Kassendaten gebracht. Edda Lorenz: „Zählt nicht Zettel – zählt, was verkauft wurde."*

BEWEISSTÜCK 1 Aggregatfunktionen verdichten Spalten

Eine **Aggregatfunktion** fasst die Werte einer Spalte über viele Zeilen zu **einem** Wert zusammen: `COUNT` (Anzahl), `SUM` (Summe), `AVG` (Durchschnitt), `MIN` (kleinster Wert), `MAX` (größter Wert). Ohne GROUP BY entsteht genau **eine** Ergebniszeile. Datenbank Stadtfest (Ausschnitt des Schemas):

`Produkt(ProduktNr, Bezeichnung, Preis, ↑StandNr)`

`Bon(BonNr, ↑StandNr, ↑HelferNr, Zeitpunkt, Zahlart)`

`Position(↑BonNr, ↑ProduktNr, Menge)`

```
SELECT COUNT(*) AS Produkte, MIN(Preis) AS Billigste,
       MAX(Preis) AS Teuerste, AVG(Preis) AS Schnitt
FROM Produkt;
```

Produkte	Billigste	Teuerste	Schnitt
26	1	6,5	3,35

BEWEISSTÜCK 2 COUNT(*), COUNT(Spalte) oder SUM(Menge)?

Die Bons 400–403 haben fünf Positionen mit den Mengen 3, 1, 3, 2 und 0 (Bon 403: Grillkäse, Menge 0 – siehe Auftrag 2):

Ausdruck	Wert	Bedeutung
`COUNT(*)`	5	Anzahl der **Zeilen** (Positionen) – auch die mit Menge 0
`SUM(Menge)`	9	Anzahl verkaufter **Stück**
`COUNT(Menge)`	5	Anzahl der Zeilen, in denen Menge **nicht NULL** ist

Steht in einer Spalte NULL (z. B. `Helfer.Verein` bei Personen ohne Verein), zählt `COUNT(Verein)` diese Zeile nicht mit, `COUNT(*)` schon. Faustregel: „Anzahl Käufe/Bons" → `COUNT(*)`, „Anzahl verkaufter Stück" → `SUM(Menge)`.

BEWEISSTÜCK 3 GROUP BY bildet Fächer

`GROUP BY Stand` sortiert alle Zeilen in **Fächer** – ein Fach je Stand. Die Aggregatfunktion wird dann **je Fach** berechnet, und jedes Fach liefert **genau eine** Ergebniszeile (Bild aus dem Menschlichen GROUP BY, eigene Daten):

1 · ALLE BON-KARTEN (FROM Bon)

Bon 101	Bon 102	Bon 104	Bon 105	Bon 106	Bon 108	Bon 109	Bon 110	Bon 113
Menge 1	Menge 2	Menge 4	Menge 1	Menge 1	Menge 3	Menge 1	Menge 3	Menge 2
Kaffeemobil	Grillhütte	Crêpes-Stand	Kaffeemobil	Grillhütte	Crêpes-Stand	Kaffeemobil	Grillhütte	Kaffeemobil

2 · GROUP BY Stand – je Stand ein Fach

FACH KAFFEEMOBIL

Bon 101	Bon 105	Bon 109	Bon 113
Menge 1	Menge 1	Menge 1	Menge 2

`COUNT(*) = 4` Zähler
`SUM(Menge) = 5` Summierer

FACH GRILLHÜTTE

Bon 102	Bon 106	Bon 110
Menge 2	Menge 1	Menge 3

`COUNT(*) = 3` Zähler
`SUM(Menge) = 6` Summierer

FACH CRÊPES-STAND

Bon 104	Bon 108
Menge 4	Menge 3

`COUNT(*) = 2` Zähler
`SUM(Menge) = 7` Summierer

3 · JE FACH GENAU EINE ERGEBNISZEILE

Stand	COUNT(*)	SUM(Menge)
Kaffeemobil	4	5
Grillhütte	3	6
Crêpes-Stand	2	7

Meiste Bons:
Kaffeemobil (COUNT)

Meiste Stück:
Crêpes-Stand (SUM)

BEWEISSTÜCK 4 Die GROUP-BY-Regel – und nach dem Schlüssel gruppieren

GROUP-BY-Regel: Jede Spalte im SELECT ist entweder **aggregiert** oder steht in **GROUP BY**. Gruppiert wird nach dem **Schlüssel**; Namen kommen zusätzlich in GROUP BY, damit sie ausgegeben werden dürfen. Beispiel: Wie viele Schichten hatte jede Person?

```
SELECT Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname,
       COUNT(*) AS Schichten
FROM Helfer, Schicht
WHERE Helfer.HelferNr = Schicht.HelferNr
GROUP BY Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname
ORDER BY Schichten DESC, Helfer.HelferNr;
```

HelferNr	Vorname	Nachname	Schichten
33	Enno	Kolthoff	4
35	Arne	Thölken	4
40	Mats	Vosgerau	4
…	…	…	…

ERMITTLER-NOTIZ Läuft ≠ richtig

SQLite führt auch Abfragen aus, die die GROUP-BY-Regel verletzen – und zeigt dann einen zufälligen Wert aus dem Fach. Im Abitur und in der Datendetektei gilt die Regel streng. Und: Am Stadtfest halfen **zwei Personen mit genau demselben Namen** – wer nur nach dem Namen gruppiert, legt sie in ein Fach.

Ermittlungsaufträge

NR.	STUFE	ORT	AUFTRAG
1	★☆☆	PAPIER	Frauke Melchers hat vier Fragen. Gib jeweils die passende Aggregatfunktion **mit Spalte** an und begründe bei b), warum COUNT hier nicht reicht.

Frage des Festausschusses	Aggregatfunktion (Spalte)
a) Wie viele Helferinnen und Helfer sind beim TuS gemeldet?	
b) Wie viele Waffeln wurden verkauft?	
c) Wann begann die früheste Schicht?	
d) Was kostet ein Produkt des Crêpes-Stands im Durchschnitt?	

NR.	STUFE	ORT	AUFTRAG

.....

2 ★☆☆ PAPIER

Die Bons 400 bis 404 enthalten diese acht Positionen. Gib das Ergebnis der Abfrage an (Reihenfolge egal). Stell dir dazu Fächer vor.

↑BonNr	↑ProduktNr	Menge
400	19	3
400	20	1
401	14	3
401	15	2
402	21	3
403	4	0
404	8	0
404	9	3

```
SELECT BonNr, COUNT(*) AS Positionen, SUM(Menge) AS Stueck
FROM Position
WHERE BonNr BETWEEN 400 AND 404
GROUP BY BonNr;
```

BonNr	Positionen	Stueck

3 ★★☆ LAB

Wie teuer ist ein Produkt im Durchschnitt – getrennt nach der **Art** des Standes (Essen, Getränke, …)? Entwirf die Abfrage, teste sie in der Konsole und gib das Ergebnis an.

.....

.....

.....

.....

4 ★★☆ PAPIER

Alle drei Abfragen laufen in SQLite. Untersuche jede auf die GROUP-BY-Regel. Markiere die Spalte, die gegen die Regel verstößt, und verändere die fehlerhaften Abfragen so, dass sie die Regel erfüllen.

a)

```
SELECT Stand.Name, Stand.Platz, COUNT(*) AS Bons
FROM Stand, Bon
WHERE Stand.StandNr = Bon.StandNr
GROUP BY Stand.StandNr, Stand.Name;
```

b)

```
SELECT Helfer.Verein, COUNT(*) AS Personen
FROM Helfer
GROUP BY Helfer.Verein;
```

c)

```
SELECT Bon.Zahlart, Bon.Zeitpunkt, COUNT(*) AS Bons
FROM Bon
GROUP BY Bon.Zahlart;
```

NR.	STUFE	ORT	AUFTRAG

.....

.....

.....

5 ★★★ PAPIER

Juri will den Umsatz je Person berechnen und schreibt die folgende Abfrage. Sie hält die GROUP-BY-Regel ein. **Erläutere**, warum sie trotzdem zu falschen Ergebnissen führen kann, und gib an, wie GROUP BY richtig lauten muss.

```
SELECT Helfer.Nachname,
       SUM(Position.Menge * Produkt.Preis) AS Umsatz
FROM Helfer, Bon, Position, Produkt
WHERE Helfer.HelferNr = Bon.HelferNr
  AND Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
GROUP BY Helfer.Nachname;
```

.....

.....

.....

.....

ABI-ECKE

Im Stil des Abiturs

Gegeben ist die Datenbank Stadtfest (Relationenschema siehe Beweisstück 1 sowie Stand(StandNr, Name, Art, Platz)). **Entwerfen** Sie eine SQL-Abfrage, die für jeden Stand die Standnummer, den Namen und die Anzahl der dort verkauften Produkte (Stückzahl) ausgibt. [4 BE]

.....

.....

.....

.....

.....

SPUR IM LAB

Ermittlung 3.1 „Kassensturz“

Aufgaben: Einstieg A3-E1-04 (Sortierfächer), dann A3-E1-01 … A3-E1-08 · Cold Case A3-E1-CC · **Werkzeug:** Sortierfächer (aus jeder Konsole aufrufbar)

COLD CASE · OHNE PUNKTE **Kartenzahler**

Welcher Anteil der Bons wurde mit Karte bezahlt? Entwirf zwei Abfragen, mit denen du das bestimmen kannst, und beschreibe, wie du aus den beiden Ergebnissen den Anteil berechnest.